

## SUMMARY

This video provides an overview of Non-Deterministic Finite Automata (NFA) within the framework of Theory of Computation. An NFA is a finite automaton that permits zero, one, or multiple transitions for a given state-input pair. Although NFAs and DFAs possess equal computational power, NFAs often offer simpler construction and reduced state complexity.

Key conclusions:

NFAs and DFAs are computationally equivalent.

An NFA with  $n$  states can generate a DFA with up to  $2^n$  states.

Subset Construction is the formal method for NFA-to-DFA conversion.

## I. FORMAL DEFINITION AND STRUCTURE

An NFA is defined as a 5-tuple:

$(Q, \Sigma, \delta, q_0, F)$

Where:

$Q$  = Finite set of states

$\Sigma$  = Input alphabet

$q_0$  = Initial state ( $q_0 \in Q$ )

$F$  = Set of final states ( $F \subseteq Q$ )

$\delta$  = Transition function

### 1. TRANSITION FUNCTION

The defining property of an NFA lies in its transition function:

$\delta : Q \times \Sigma \rightarrow 2^Q$

Here,  $2^Q$  represents the power set of  $Q$ .

This allows:

Zero transitions (mapping to  $\emptyset$ )

Multiple transitions (mapping to a subset of states)

Unlike a DFA, which maps to exactly one state, an NFA maps to a set of states.

## II. OPERATIONAL LOGIC AND CHARACTERISTICS

### 1. ACCEPTANCE CRITERIA

A string  $W$  is accepted if:

There exists at least one transition path from  $q_0$  to any state in  $F$  after reading the entire string.

A string is rejected if:

All paths either terminate early or end in non-final states.

## 2. RESPONSE TO INPUTS

Valid Inputs:

NFAs only require transitions for valid strings.

Invalid Inputs:

No obligation exists to define transitions for invalid strings. Therefore, NFAs do not require explicit dead states.

## 3. COMPARISON OF MINIMAL FORMS

| Feature                 | DFA                          | NFA                             |
|-------------------------|------------------------------|---------------------------------|
| Minimal Form Uniqueness | Unique minimal DFA           | Multiple minimal NFAs possible  |
| Computational Power     | Equivalent to NFA            | Equivalent to DFA               |
| Construction Complexity | Must define every transition | Only valid transitions required |

## III. NFA CONSTRUCTION PATTERNS

NFAs focus on constructing only successful paths, making design intuitive.

### 1. STANDARD PATTERNS ( $\Sigma = \{a, b\}$ )

| Pattern                         | Minimal States (NFA) |
|---------------------------------|----------------------|
| Starts with 'a'                 | 2                    |
| Ends with 'a'                   | 2                    |
| Contains substring "ab"         | 3                    |
| $n^{\text{th}}$ symbol from LHS | $n + 1$              |
| $n^{\text{th}}$ symbol from RHS | $n + 1$              |

Note:

For RHS positional problems, DFA requires  $2^n$  states, whereas NFA requires only  $n+1$ .

## 2. NUMERICAL AND MODULAR CONSTRAINTS

| Constraint                          | Minimal States (NFA) |
|-------------------------------------|----------------------|
| Exactly $n$ occurrences             | $n + 1$              |
| At most $n$                         | $n + 1$              |
| At least $n$                        | $n + 1$              |
| Modulo $n$ (length or symbol count) | $n$                  |

In many counting problems, NFA construction mirrors DFA construction.

## IV. NFA TO DFA CONVERSION (SUBSET CONSTRUCTION)

Subset Construction systematically converts an NFA into an equivalent DFA.

### 1. CONVERSION ALGORITHM

Step 1: The DFA initial state equals the NFA initial state.

Step 2: For each DFA state (a set of NFA states) and input symbol, compute the union of all reachable NFA states.

Step 3: Continue generating new state sets until no new sets appear.

Step 4: Mark as final any DFA state containing at least one NFA final state.

### 2. STATE COMPLEXITY BOUNDARIES

If an NFA has  $n$  states:

$$1 \leq m \leq 2^n$$

Where  $m$  is the number of DFA states after conversion.

Best Case:

1 state (e.g., accepting  $\Sigma^*$ )

Worst Case:

$2^n$  states (common in RHS positional constraints)

### 3. MAXIMUM NUMBER OF FINAL STATES AFTER CONVERSION

If:

Total NFA states =  $n$

Non-final NFA states =  $n_{\text{non-final}}$

Maximum possible DFA final states:

Max Final States =  $2^n - 2^{n_{\text{non-final}}}$

Example:

Total NFA states = 10

Final states = 3

Non-final states = 7

Max DFA final states:

$$\begin{aligned} &2^{10} - 2^7 \\ &= 1024 - 128 \\ &= 896 \end{aligned}$$

## V. CRITICAL INSIGHTS

Equivalence:

Every DFA is technically an NFA.

Every NFA can be converted into an equivalent DFA.

Dead State vs.  $\emptyset$ :

In DFA, a dead state is an explicit state with self-loops.

In NFA, absence of transition ( $\emptyset$ ) represents termination of that path.

Non-Minimal Conversion:

Subset construction does not guarantee minimal DFA.

Further minimization may be required.

Path Determinism:

DFA: Exactly one computation path per string.

NFA: Multiple computation paths may exist; acceptance requires only one successful path.